

AI Coding Tool Adoption: Complete Project Ideas Catalog

Network Science Project Planning Document

February 2026

Contents

1	Executive Summary	2
2	Core Network Science Concepts Applicable	2
2.1	Information Cascades & Contagion	2
2.2	Social Influence vs. Homophily	2
2.3	Network Centrality & Influence	2
2.4	Community Structure	3
2.5	Multiplex & Multilayer Networks	3
2.6	Temporal Network Dynamics	3
2.7	Bipartite Networks	3
3	Project Ideas: Full Catalog	4
3.1	Tier 1: Flagship Projects (Recommended)	4
3.2	Tier 2: Strong Alternatives	5
3.3	Tier 3: Focused Studies	6
3.4	Tier 4: Advanced/Experimental	7
4	Recommended Project Combinations	7
4.1	Option A: Cascade Focus (Theoretical + Empirical)	7
4.2	Option B: Causal Inference Focus (Methodological)	8
4.3	Option C: Ecosystem Focus (Multi-Tool)	8
5	Technical Implementation Notes	8
5.1	Network Construction	8
5.2	Temporal Analysis	8
5.3	Key Libraries	8
6	Quick Reference: Network Metrics to Compute	9
7	References	9

1 Executive Summary

This document catalogs all potential network science project directions using our unique dataset of 5+ million AI-assisted code contributions. The dataset captures adoption of four major AI coding tools across 600,000+ GitHub repositories from February 2025 to January 2026.

Dataset at a Glance

- **5.08 million** AI-assisted contributions (commits + PRs)
- **600,000+** unique repositories
- **4 AI tools:** Claude Code, Copilot, Codex, Jules
- **500,000+** estimated unique developers
- **11 months** of temporal data

2 Core Network Science Concepts Applicable

2.1 Information Cascades & Contagion

- **Independent Cascade Model:** Each “infected” developer has probability p of infecting each neighbor
- **Linear Threshold Model:** Developers adopt when fraction of adopting neighbors exceeds threshold θ
- **SIR/SIS Models:** Susceptible-Infected-Recovered dynamics for tool adoption
- **Bass Diffusion:** Innovation adoption with internal (peer) and external (marketing) influence

Application: Model how Claude Code usage spreads from developer to developer through collaboration networks.

2.2 Social Influence vs. Homophily

- **Peer Effects:** Do developers adopt tools their collaborators use?
- **Homophily:** Do similar developers (same language/domain) independently choose similar tools?
- **Confounding:** Do shared environments (companies) drive both?

Application: Causal inference to separate influence from selection effects in AI tool adoption.

2.3 Network Centrality & Influence

- **Degree Centrality:** Do highly connected developers adopt faster?
- **Betweenness:** Do bridge developers spread tools across communities?
- **PageRank:** Who are the influential early adopters?

Application: Identify which network positions predict early adoption.

2.4 Community Structure

- **Modularity-based Detection:** Louvain, Girvan-Newman
- **Information-theoretic:** Infomap algorithm
- **Overlapping Communities:** Developers belonging to multiple groups

Application: Do communities form around specific AI tools or programming languages?

2.5 Multiplex & Multilayer Networks

- **Layer Correlation:** Same developers central across tools?
- **Cross-layer Influence:** Does Copilot adoption predict Claude adoption?
- **Layer Switching:** Migration patterns between tools

Application: Model the AI tool ecosystem as competing/complementary layers.

2.6 Temporal Network Dynamics

- **Burst Detection:** When do adoption events cluster?
- **Temporal Motifs:** Patterns of sequential adoption
- **Network Evolution:** How does the collaboration graph change?

Application: Analyze adoption dynamics over our 11-month window.

2.7 Bipartite Networks

- **Developer-Repository Graph:** Natural structure of our data
- **One-mode Projections:** Developer-Developer and Repo-Repo networks
- **Bipartite Community Detection:** Specialized algorithms

Application: The fundamental network structure connecting developers to codebases.

3 Project Ideas: Full Catalog

3.1 Tier 1: Flagship Projects (Recommended)

Project 1A: Information Cascades in AI Tool Adoption

Research Question: Does AI tool adoption spread through developer networks like a cascade?

Methodology:

1. Build collaboration network from shared repository contributions
2. Extract temporal adoption sequences for each tool
3. Fit Independent Cascade model; estimate transmission parameter β
4. Compare cascade sizes/depths across tools
5. Predict future adoptions from network state

Key Deliverables:

- Cascade tree visualizations
- Calibrated epidemic model
- Adoption prediction model

Network Concepts: Information cascades, SIR models, temporal networks

Project 1B: Influence vs. Homophily in Tool Selection

Research Question: Is tool clustering due to peer influence or similar developers making similar choices?

Methodology:

1. Temporal regression: Does neighbor adoption at $t - 1$ predict own adoption at t ?
2. Matched pair design: Compare exposed vs. unexposed developers
3. Network randomization: Permutation tests preserving structure

Key Deliverables:

- Decomposition of influence vs. homophily contributions
- Comparison across tools (which shows strongest network effects?)
- Causal inference framework for technology adoption

Network Concepts: Peer effects, causal inference, homophily

3.2 Tier 2: Strong Alternatives

Project 2A: Influence Maximization for Developer Outreach

Research Question: Which k developers should be targeted to maximize adoption through network effects?

Methodology:

1. Implement greedy influence maximization (Kempe et al. 2003)
2. Compare with degree, PageRank, and betweenness heuristics
3. Validate on historical data: Did actual early adopters maximize spread?

Key Deliverables:

- Ranked list of influential developers for each tool
- Algorithm comparison and recommendations
- Expected ROI from targeted outreach

Network Concepts: Influence maximization, centrality, viral marketing

Project 2B: Multi-Tool Ecosystem as Multiplex Network

Research Question: How do developers navigate competing AI tools? Does adopting one predict adopting/rejecting others?

Methodology:

1. Link developers across tool datasets
2. Build multiplex network with one layer per tool
3. Compute layer correlations and cross-layer influence
4. Model tool switching as Markov process

Key Deliverables:

- Tool ecosystem map with migration flows
- Transition probability matrix
- Market share predictions
- Developer segmentation (loyalists vs. explorers)

Network Concepts: Multiplex networks, layer correlation, Markov chains

Project 2C: Community Detection and Tool Preference Clustering

Research Question: Do distinct communities form around different AI tools?

Methodology:

1. Project bipartite graph to developer-developer network
2. Apply Louvain, Infomap, label propagation
3. Characterize communities by language, topics, location
4. Correlate community membership with tool preference

Key Deliverables:

- Community map colored by dominant tool
- Community profiles (characteristics)
- Predictive model: community $\rightarrow$ tool adoption

Network Concepts: Community detection, modularity, bipartite networks

3.3 Tier 3: Focused Studies

Project 3A: Geographic Diffusion of AI Tools

Research Question: How does adoption spread geographically? Are there regional leader-follower dynamics?

Method: Spatial network from cross-region collaborations; spatial epidemic model.

Deliverables: Animated world map of adoption spread; regional adoption curves.

Project 3B: Programming Language Networks and Tool Preference

Research Question: Do Python developers prefer different tools than JavaScript developers?

Method: Build language co-occurrence network; correlate with tool adoption rates.

Deliverables: Language-tool affinity matrix; cross-language diffusion analysis.

Project 3C: Repository Adoption Dynamics

Research Question: What predicts whether a repository will adopt AI tools?

Method: Repository feature analysis (stars, forks, language, age); network of related repos.

Deliverables: Adoption prediction model; repository characteristics associated with AI usage.

Project 3D: Temporal Burst Analysis

Research Question: When do AI tool adoption events cluster in time? Are there triggering events?

Method: Burst detection algorithms; correlation with external events (releases, announcements).

Deliverables: Timeline of adoption bursts; event-adoption correlation analysis.

Project 3E: Bot vs. Human Collaboration Networks

Research Question: How does AI (as a “collaborator”) change the structure of development networks?

Method: Compare pre/post AI tool adoption network metrics; treat AI as pseudo-node.

Deliverables: Network structural changes from AI integration; human-AI collaboration patterns.

3.4 Tier 4: Advanced/Experimental

Project 4A: Game-Theoretic Model of Tool Competition

Research Question: Can we model AI tool adoption as a coordination game on networks?

Method: Network games with switching costs; equilibrium analysis.

Deliverables: Game-theoretic model; predicted equilibrium market shares.

Project 4B: Link Prediction for Future Collaborations

Research Question: Can AI tool co-usage predict future collaborations?

Method: Link prediction using tool adoption as features.

Deliverables: Link prediction model; analysis of tool-collaboration relationship.

Project 4C: Network Embedding for Developer Similarity

Research Question: Can we learn developer embeddings that capture tool preferences?

Method: Node2Vec, GraphSAGE on collaboration network.

Deliverables: Developer embeddings; visualization of tool preference clusters in embedding space.

4 Recommended Project Combinations

4.1 Option A: Cascade Focus (Theoretical + Empirical)

1. **Project 1A** (Cascades) as primary
2. **Project 2A** (Influence Maximization) as secondary

Story: “We model how AI tools spread through networks, then identify optimal seed developers.”

4.2 Option B: Causal Inference Focus (Methodological)

1. **Project 1B** (Influence vs. Homophily) as primary
2. **Project 3A** (Geographic) as secondary

Story: “We rigorously identify peer influence effects, then analyze geographic dimensions.”

4.3 Option C: Ecosystem Focus (Multi-Tool)

1. **Project 2B** (Multiplex) as primary
2. **Project 2C** (Communities) as secondary

Story: “We map the AI tool ecosystem and identify distinct developer communities.”

5 Technical Implementation Notes

5.1 Network Construction

Developer Collaboration Network:

```
# Pseudocode
for each repository R:
    devs = developers who contributed to R
    for each pair (d1, d2) in devs:
        G.add_edge(d1, d2, weight=1) # or increment weight
```

Edge weights: Number of shared repos, or recency-weighted.

5.2 Temporal Analysis

Adoption time extraction:

```
adoption_time[dev] = min(commit_date) for commits by dev using tool
```

Cascade reconstruction: For each adopter, assign “parent” as the neighbor who adopted most recently before them.

5.3 Key Libraries

- **networkx** or **igraph**: Graph operations
- **ndlib**: Epidemic/cascade simulations
- **community** (python-louvain): Community detection
- **geopandas**: Geographic analysis
- **pyvis** or **gephi**: Visualization

Metric	Interpretation
Degree	Number of collaborators
Clustering coefficient	Local cliquishness
Betweenness centrality	Bridge position
PageRank	Influence/importance
Modularity	Community structure quality
Average path length	Network compactness
Giant component size	Network connectivity
Assortativity	Degree correlation

Table 1: Standard network metrics for developer collaboration graph

6 Quick Reference: Network Metrics to Compute

7 References

1. Kempe, Kleinberg, & Tardos (2003). “Maximizing the spread of influence.” KDD.
2. Aral, Muchnik, & Sundararajan (2009). “Distinguishing influence from homophily.” PNAS.
3. Centola (2010). “Spread of behavior in online social networks.” Science.
4. Bakshy et al. (2012). “Role of social networks in information diffusion.” WWW.
5. Leskovec et al. (2007). “Dynamics of viral marketing.” TWEB.
6. Rogers (1962). *Diffusion of Innovations*. Free Press.
7. Watts & Strogatz (1998). “Collective dynamics of small-world networks.” Nature.
8. Barabasi & Albert (1999). “Emergence of scaling in random networks.” Science.